Foundations of AI & ML **Fall 2026**
Professor Amy Greenwald[1] **Reinforcement Learning Unit**

Markov Processes & Dynamic Programming, cont'd

In the 1950's, mathematician Richard Bellman pioneered the principles of dynamic programming (DP). DP applies to problems with recursive substructure and overlapping subproblems, in which case: (i) the problem breaks down into more than one smaller version of the exact same problem, and (ii) the smaller problems overlap. The canonical example the Fibonacci numbers, i.e., 1, 1, 2, 3, 5, 8, 13, etc., defined as $F(n) = F(n-1) + F(n-2)$. Computing $F(n)$ involves $F(n-2)$ twice, i.e., $F(n-2) + F(n-3) + F(n-2)$. In a top-down approach to DP (i.e., memoization), this intermediate value is cached and accessed as needed, rather than computed twice. In a bottom-up approach, we start from the base cases, i.e., $F(0) = F(1) = 1$, and work from there, e.g., $F(2) = F(1) + F(0)$, using the requisite values as they are already available.

1 Bellman's Theorem and Equations

We begin by deriving Bellman's theorem for Markov reward processes: the state value $V(s_t)$ at state s_t is the sum of the immediate reward obtained at time t and the discounted sum of the expected rewards obtained thereafter: i.e., for $0 \leq \gamma < 1$,

$$V(s_t) = r_t + \gamma \mathbb{E}_{s_{t+1}}[V(s_{t+1})] \tag{1}$$

The crux of the proof of this theorem is roughly as follows: for all states $s \in \mathcal{S}$,

$$\begin{aligned}
V(s) &= \mathbb{E}_\tau \left[G_t^\tau \mid S_t = s \right] \\
&= \int_\tau P[\tau \mid S_t = s] G_t^\tau \\
&= r_t + \gamma \int_\tau P[\tau \mid S_t = s] G_{t+1}^\tau \\
&= r_t + \gamma \sum_{s' \in \mathcal{S}} P[S_{t+1} = s' \mid S_t = s] \left(\int_{\tau'} P[\tau' \mid S_{t+1} = s', S_t = s] G_{t+1}^{\tau'} \right) \\
&= r_t + \gamma \sum_{s' \in \mathcal{S}} P[S_{t+1} = s' \mid S_t = s] \left(\int_{\tau'} P[\tau' \mid S_{t+1} = s'] G_{t+1}^{\tau'} \right) \\
&= r_t + \gamma \sum_{s' \in \mathcal{S}} P[S_{t+1} = s' \mid S_t = s] \mathbb{E}_{\tau'} \left[G_{t+1}^{\tau'} \mid S_{t+1} = s' \right] \\
&= r_t + \gamma \sum_{s' \in \mathcal{S}} P[S_{t+1} = s' \mid S_t = s] V(s') \\
&= r_t + \gamma \mathbb{E}_{s'}[V(s')]
\end{aligned}$$

The first equality follows from the definition of V; the second, from the meaning of an expectation; the third, from the fact that r_t is not a random quantity; the fourth, from the definition of joint probability; the fifth, from the Markov property; the sixth, from the meaning of an expectation (again); the seventh, from the definition of V (again); and the last line is simply an abbreviation.

[1] These notes were compiled in conjunction with Professor Eric Ewing.

Bellman's theorem gives rise to the following system of $|S|$ equations with $|S|$ unknowns, known as Bellman's equations: for all states $s \in \mathcal{S}$,

$$V(s) = r(s) + \gamma \sum_{s'} P[s' \mid s] V(s') \tag{2}$$

Assuming a finite state state, with n states, we can rewrite this equation in matrix notation as follows: for value and reward vectors $v, r \in \mathbb{R}^n$ and $n \times n$ transition probability matrix P,

$$v = r + \gamma P v \tag{3}$$
$$v - \gamma P v = r \tag{4}$$
$$Iv - \gamma P v = r \tag{5}$$
$$(I - \gamma P)v = r \tag{6}$$
$$v = (I - \gamma P)^{-1} r \tag{7}$$

Therefore, Bellman's equation can be solved by matrix inversion. But just as in linear regression, where we tend to prefer running gradient descent to computing a closed-form solution directly, iterative methods are the norm for solving Bellman's equations. Iterative methods are preferable when the system of equations is large—making matrix inversion expensive not only in terms of time, but even more so in terms of space—and because iterative methods are more robust to numerical approximation errors.

2 Policy Evaluation

Policy evaluation is a dynamic programming method that computes state values via iterative updates based on Bellman's equations:

$$V(s) \leftarrow r(s) + \gamma \sum_{s'} P[s' \mid s] V(s') \tag{8}$$

This update rule is described in pseudocode in Table 1.

POLICY_EVALUATION(MRP, γ, ϵ)	
Inputs	discount factor γ
	convergence test ϵ
Output	state-value function V
Initialize	$V = 0$ and $V' = \infty$

while $\max_s |V(s) - V'(s)| > \epsilon$ **do**

1. $V' = V$
2. for all $s \in \mathcal{S}$
   (a) $V(s) = r(s) + \gamma \sum_{s'} P[s' \mid s] V'(s')$

return V

Table 1: Policy Evaluation.

Gauss-Seidel's version of this algorithm incorporates in-place updating: i.e., updating with V, as shown in Table 2, rather than V', as shown in Table 1.

Let's run through these algorithms on the Gambler's Ruin, assuming rewards 0 everywhere except at state N, where the reward is 1.

GAUSS_SEIDEL(MRP, γ, ϵ)	
Inputs	discount factor γ
	convergence test ϵ
Output	state-value function V
Initialize	$V = 0$ and $V' = \infty$

`while` $\max_s |V(s) - V'(s)| > \epsilon$ `do`

1. $V' = V$
2. for all $s \in \mathcal{S}$
   (a) $V(s) = r(s) + \gamma \sum_{s'} P[s' \mid s] V(s')$

`return` V

Table 2: Gauss-Seidel.

Assuming $\gamma = 1$, these values can be computed via policy evaluation as follows:

V	0	1	2	3	4	END
0	0	0	0	0	0	0
1	0	0	0	0	1	0
2	0	0	0	$\frac{1}{3}$	1	0
3	0	0	$\frac{1}{9}$	$\frac{1}{3}$	1	0
4	0	$\frac{1}{27}$	$\frac{1}{9}$	$\frac{11}{27}$	1	0
5	0	$\frac{1}{27}$	$\frac{13}{81}$	$\frac{11}{27}$	1	0

With in-place computation *á la* Gauss-Seidel, working backwards from END to state 4 down to state 0, the computation proceeds as follows:

V	0	1	2	3	4	END
0	0	0	0	0	0	0
1	0	$\frac{1}{27}$	$\frac{1}{9}$	$\frac{1}{3}$	1	0
2	0	$\frac{13}{243}$	$\frac{13}{81}$	$\frac{11}{27}$	1	0
3	0	$\frac{133}{2187}$	$\frac{133}{729}$	$\frac{107}{243}$	1	0
100	0	0.0667	0.2	0.4667	1	0

Since rewards are 0 everywhere unless the gambler is *not* ruined, in which case rewards are 1, the final state values can be interpreted as the probability that the gambler is *not* ruined. The value function reveals that the gambler is more likely to be ruined than not at all states $1, \ldots, N-1$.

A Convergence of Policy Evaluation

In this appendix, we show that Bellman's system of equations (Equation 2) has a solution, and that policy evaluation converges to this solution.

To solve Bellman's equations, we rely on Banach's fixed point theorem, also called the contraction mapping theorem. Given a metric space[2] (X, d), a mapping $f : X \to X$ is called a **contraction** iff there exists some $0 \leq k < 1$ *s.t.* $d(f(x), f(y)) \leq kd(x, y)$, for all $x, y \in X$.

Theorem [Banach] Given a complete[3] metric space (X, d) and a contraction mapping $f : X \to X$,

(i) there exists a unique $x^* \in X$ *s.t.* $f(x^*) = x^*$

(ii) for arbitrary $x^0 \in X$, the sequence $\{x^n\}$ defined by $x^{n+1} = f(x^n) = f^{n+1}(x^0)$ converges to x^*

To solve Bellman's equations, we define the mapping $f : \mathbb{R}^S \to \mathbb{R}^S$ as follows:

$$(f(x))(s) = r(s) + \gamma \sum_{s'} P[s' \mid s]x(s') \tag{9}$$

Theorem The mapping f is a contraction on $(\mathbb{R}^S, L_\infty)$.

[2] A metric space (X, d) is a set X together with a distance function $d : X \times X \to \mathbb{R}$ that satisfies: (i) $d(x, x) = 0$, for all $x \in X$; (ii) $d(x, y) = d(y, x)$ for all $x, y \in X$; and (iii) the triangle inequality—$d(x, z) \leq d(x, y) + d(y, z)$ for all $x, y, z \in X$.

[3] An example of a complete metric space is $\mathbb{R}$.

Proof Let the metric d be the L_∞, or max, norm: i.e., $||x - y|| = \max_i |x_i - y_i|$. For all $x, y \in X$, and for arbitrary state $s \in \mathcal{S}$,

$$\begin{aligned}
& |(f(x))(s) - (f(y))(s)| \\
= & \left| r(s) + \gamma \sum_{s'} P[s' \mid s] x(s') - \left(r(s) + \gamma \sum_{s'} P[s' \mid s] y(s') \right) \right| \\
= & \gamma \sum_{s'} P[s' \mid s] \, |x(s') - y(s')| \\
\leq & \gamma \sum_{s'} P[s' \mid s] \max_{s''} |x(s'') - y(s'')| \\
= & \gamma \sum_{s'} P[s' \mid s] ||x - y|| \\
= & \gamma ||x - y||
\end{aligned}$$

It follows that $|(f(x))(s) - (f(y))(s)| \leq \gamma ||x - y||$, for all states s. Therefore, $||f(x) - f(y)|| = \max_s |(f(x))(s) - (f(y))(s)| \leq \gamma ||x - y||$.

Corollary Bellman's system of equations (Equation 2) indeed has a fixed point solution, and the iterative application of f converges to this solution.